Probecard V3 User manual

Section1. Board description

Probecard_v3 consists of mother board and daughter board as shown in figure 1~3. Mother board has most of probecard functions and daughter board supplies necessary voltages to mother board. Daughter board has also USB3 interface to communicate with PC. Two boards are connected with two 50 pin headers and sockets on the corner side. User should be careful not to bend header pins when separating daughter board from mother board.

<Connectors, jumpers, switch and LEDs>

Please find numbering in the figure 1 and 2.

1-1 LEDs showing needle contact status. (Please refer to section 2. Operate probecard.)

1-2 When this jumper installed, AGND will be shorted to probecard's ground.

This one should be installed when using standalone mode, otherwise removed.

1-3 Left 14 pin connector is for CPLD JTAG programming.

1-4 Middle one is for FPGA JTAG port access. Programming FPGA using USB3 interface is recommended though.

1-5 Right one is for debugging purpose. Currently pin #1 outputs 40 MHz clock and pin #2 goes high when onboard clock synthesizer is locked. Other pins are not programmed yet.

2-1 Power adapter connector for board's main power. Use +5 Vdc. Adapter with current capacity more than 5 A is recommended.

2-2 This switch when ON connect USB3 MCU's GPIO pins to RS-232 interface.

2-3 Install this jumper only for emergency boot. Emergency boot is only necessary when user firmware fails connecting to USB3. (Please refer to 3.Firmware section.)

2-4 USB3 connector. Connect only USB3 cable. USB2 cable will fail proper operation.

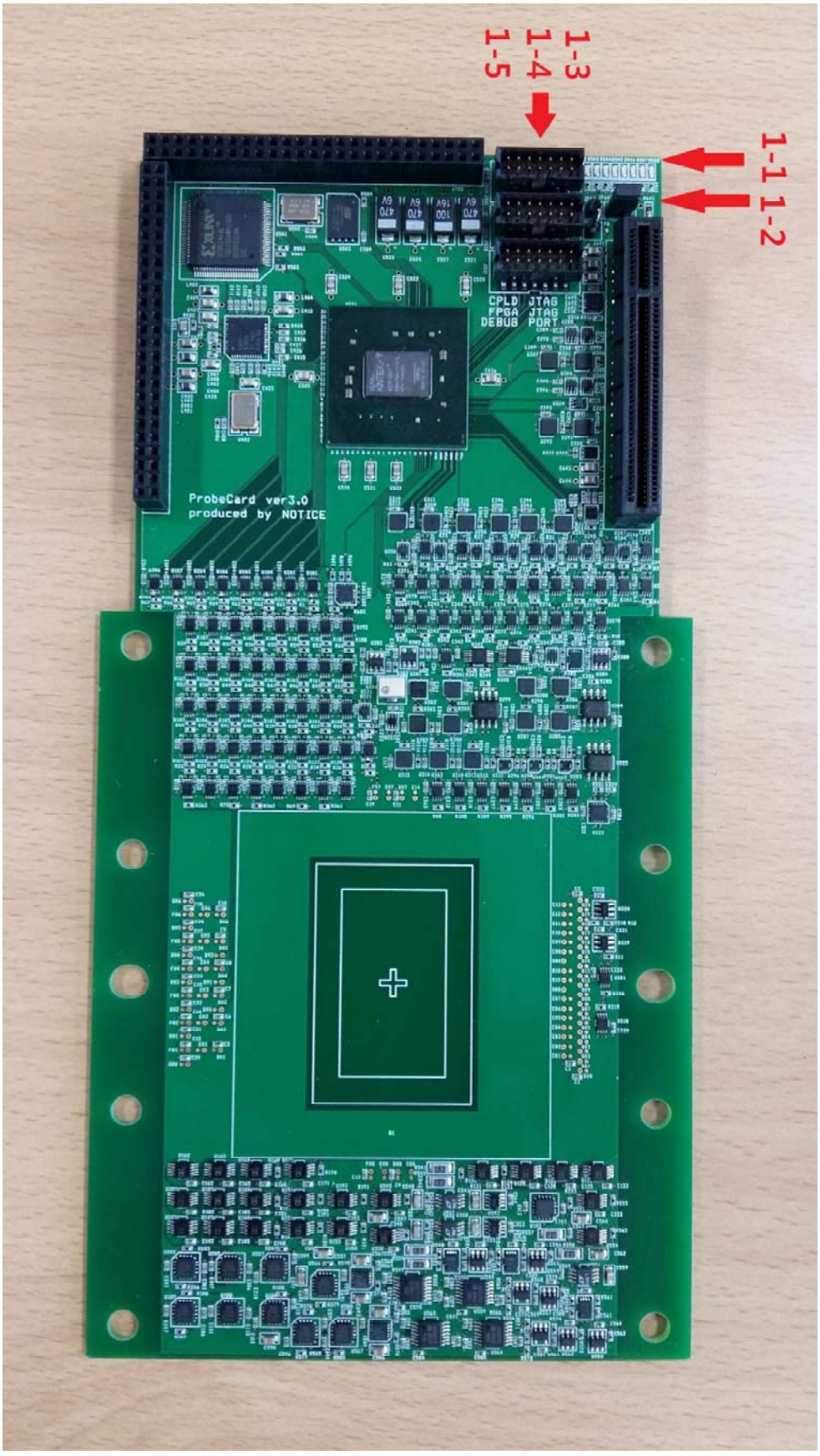


Figure 1 . Mother board

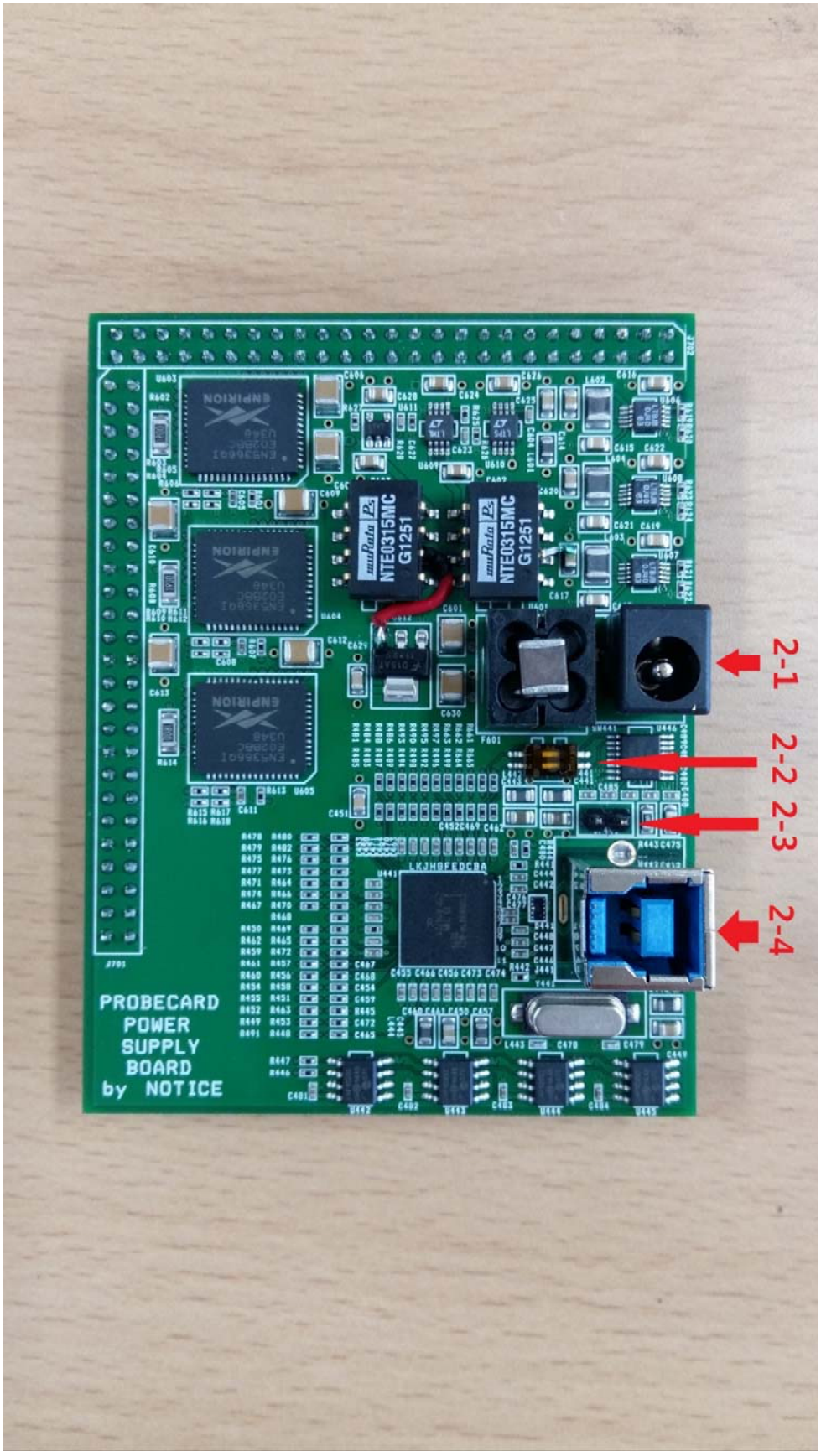


Figure 2. Daughter board

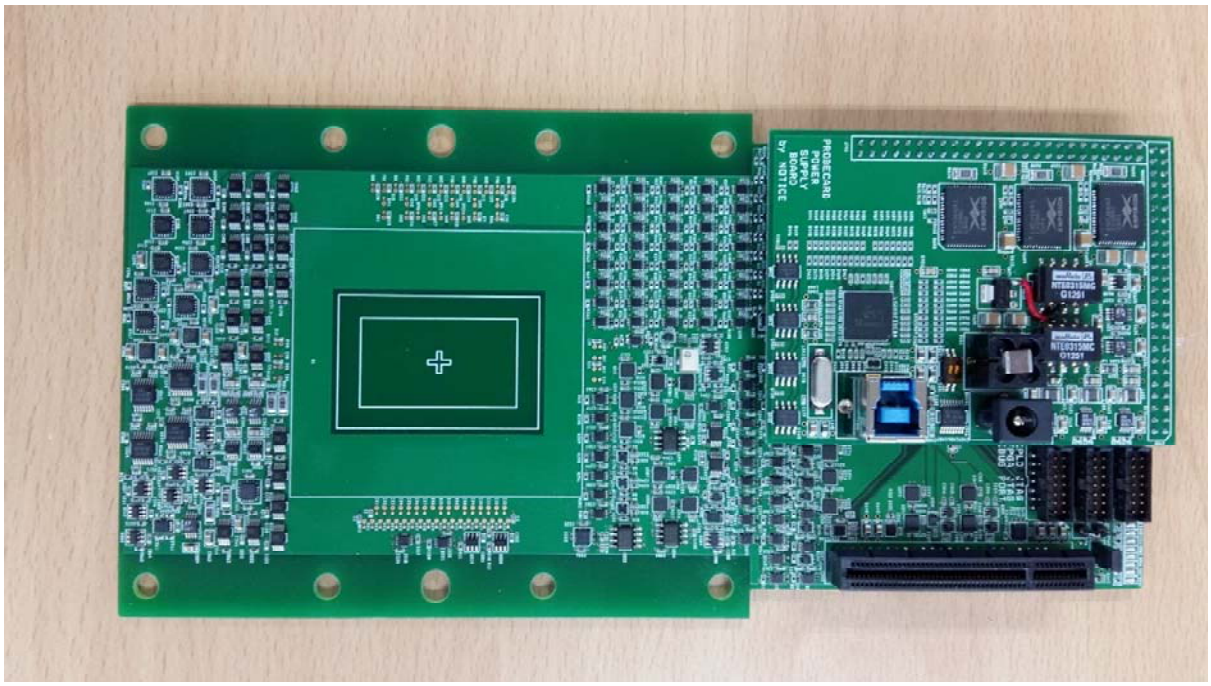


Figure 3. Mother board + daughter board

Section 2. Operate probecard

1. Connect daughter board to mother board as shown in figure 3.
2. Connect USB3 cable. This should be done before step 3.
3. Connect +5 Vdc power.
4. 3~4 seconds after power-up, probecard can be operated. When power on all LEDs are ON and after probecard's booting is finished all LEDs are turned OFF.
5. After booting, probecard should be detected as super speed USB device.

   (vendor id = 0x0547, product id = 0xCE03)
6. There are 4 modes of operation.

   A. Manual contact test mode

      Contact probing and sensing are done by software. (Please refer to section 4. Software.)

   B. Auto contact test mode

      Contact probing and sensing are done by MCU. MCU perform contact test and indicates test status with LEDs. User can read the status by software too. LEDs from up to down in figure 3 represent :

      LED0 : All AVDD pads are contacted.

      LED1 : All DVDD pads are contacted.

LED2 : All AVSS pads are contacted.

LED3 : All DVSS pads are contacted.

LED4 : DVDD33 pad is contacted.

LED5 : All PWELL pads are contacted.

LED6 : All SUB pads are contacted.

LED7 : All signal pads are contacted.

C. DAQ mode

In this mode, probecard is disconnected from chip and passes all signals to PCI connector.

D. Standalone mode

Same as DAQ mode except voltage to chip is supplied by probecard rather than by PCI connector. (Full standalone DAQ functionality will be implemented later.)

Section 3. Firmware

3 kinds of firmwares are necessary for probecard. One is CPLD firmware, another is USB3 MCU's firmware and last one is FPGA firmware.

1. CPLD firmware

   A Xilinx CPLD(XC95144XL) is used for uploading/downloading FPGA firmware and programming clock synthesizer. Firmware can be downloaded using JTAG port and Xilinx's IMPACT software. Because CPLD's job is almost fixed, reprogramming CPLD by user is not expected often.

2. USB3 MCU's firmware

   Cypress CYUSB3014 device is used for USB interface. This firmware does three things :

   A. USB device setup including slave FIFO configuration.

   B. Uploading FPGA firmware and programming clock synthesizer via CPLD.

   C. Auto contact mode operation.

3. FPGA firmware

   FPGA(Xilinx XC7K160) firmware is stored in 64 Mbit serial flash memory. Uploading firmware can be done using USB3 interface.

<How to upload MCU/FPGA firmware>

Use update_probecard_v3.exe software at probecardv3/programmer. To run this one libusb-1.0.0 should be installed on user's PC. Update_probecard_v3.exe has following functions :

1. ./update_probecard_v3.exe –list

   This lists all USB devices attached on user's PC. Here one sees device_id something like BB:AA where BB is USB bus number and AA is USB address.

2. ./update_probecard_v3.exe –dev device_id –info

   This shows probecard information. If you enter "all" instead of "device_id" you'll see every attached device's information. The information shown includes FPGA firmware file name and board sid which is uniquely assigned to a board for application using multiple probecards. (Even you are using one probecard, you need sid to access probecard.)

3. ./update_probecard_v3.exe –dev device_id –sid <sid>

   This will set sid for probecard.

4. ./update_probecard_v3.exe –dev device_id –fpga <firmware filename>

   This uploads FPGA firmware to 64 Mbit flash memory. Firmware filename is stored at the same time.

5. ./update_probecard_v3.exe –dev device_id –fx3 <firmware filename>

   This uploads USB3 MCU firmware to serial PROM. Serial PROM has 512 kbytes memory space.

<How to compile update_probecard_v3.exe>

1. Before compiling, you need libusb-1.0.0 and libusb-1.0.0-dev installed.
2. Go to probecardv3/programmer
3. In Makefile, modify libusb library path(where you install libusb-1.0.0) if necessary.
4. make

<Emergency USB3 MCU firmware uploading>

1. For FPGA uploading, even this FPGA firmware doesn't work at all, you can upload firmware at any time(even after this wrong FPGA firmware is already downloaded.)
2. But if similar thing happens when uploading USB3 MCU firmware, you may loose connection to USB3 then update_probecard_v3.exe doesn't work. In this case, install a jumper on 2-3 header shown in figure 2, and recycle power. Then MCU boot with its built-in bootloader and by using programming software provided by Cypress(cyusb_linux in Linux PC) you can recover the probecard.

Section 4. Software

There are two software code provided. One is probecard's function library and the other is sample user code.

1. To compile library

   Go to probecardv3/software/src

   make

   make install

2. Library is a set of read/write functions of probecard registers. Followings are probecard registers.

Address	Register
0	Probe/sense AVSS, AVDD, DVSS and DVDD
1	Probe/sense DVDD33, PWELL and SUB
2	Probe/sense signal pads
3	Set voltage/read current AVDD and DVDD
4	Read current DVDD33
5	Set voltage/read current PWELL and SUB
6	Short combination AGND, PWELL and SUB
7	Read DACMONV and DACMONI voltage
8	Operation mode
9	DATA and BUSY buffer direction
A	Start ADC conversion for monitoring
B	LED
C	Clock synthesizer lock status
D	Reserved
E	Reserved
F	Dummy register for readback test

<Probe/sense AVSS, AVDD, DVSS and DVDD>

Address = 0x0

D31	D30	D29	D28	D27	D26	D25	D24
DVDD7	DVDD6	DVDD5	DVDD4	DVDD3	DVDD2	DVDD1	DVDD0
D23	D22	D21	D20	D19	D18	D17	D16
DVSS7	DVSS6	DVSS5	DVSS4	DVSS3	DVSS2	DVSS1	DVSS0
D15	D14	D13	D12	D11	D10	D9	D8
AVDD7	AVDD6	AVDD5	AVDD4	AVDD3	AVDD2	AVDD1	AVDD0
D7	D6	D5	D4	D3	D2	D1	D0
AVSS7	AVSS6	AVSS5	AVSS4	AVSS3	AVSS2	AVSS1	AVSS0

When write to 1, probing CMOS buffer output is enabled.

When read, 1 is returned if contact is made, 0 is returned if contact is not made.

< Probe/sense DVDD33, PWELL and SUB>

Address = 0x1

D31	D30	D29	D28	D27	D26	D25	D24
-	-	-	-	-	-	-	-
D23	D22	D21	D20	D19	D18	D17	D16
-	-	-	-	-	-	-	-
D15	D14	D13	D12	D11	D10	D9	D8
-	-	-	-	-	-	SUB4	SUB3
D7	D6	D5	D4	D3	D2	D1	D0
SUB2	SUB1	SUB0	PWELL3	PWELL2	PWELL1	PWELL0	DVDD33

When write to 1, probing CMOS buffer output is enabled.

When read, 1 is returned if contact is made, 0 is returned if contact is not made.

< Probe/sense signal pads >

Address = 0x2

D31	D30	D29	D28	D27	D26	D25	D24
-	-	-	-	DACMONI	DACMONV	MCLK_N	MCLK_P
D23	D22	D21	D20	D19	D18	D17	D16
DCTRL_N	DCTRL_P	DCLK_N	DCLK_P	HSDATA_N	HSDATA_P	RST_N	POR_DIS
D15	D14	D13	D12	D11	D10	D9	D8
CHIPID6	CHIPID5	CHIPID4	CHIPID3	CHIPID2	CHIPID1	CHIPID0	CTRL
D7	D6	D5	D4	D3	D2	D1	D0
BUSY	MISTRAL-O2	MISTRAL-O1	MISTRAL-O0	DATA3	DATA2	DATA1	DATA0

When write to 1, probing CMOS buffer output is enabled.

When read, 1 is returned if contact is made, 0 is returned if contact is not made.

< Set voltage/read current AVDD and DVDD >

Address = 0x3

When write to register

D31-18	D17	D16-D9	D8	D7-0
-	Arm DVDD	DVDD V	Arm AVDD	AVDD V

AVDD V = potentiometer value for adjusting onboard AVDD regulator

Arm AVDD = only when 1 is written to this, AVDD regulator is set,

When 0 is written, AVDD voltage is not changed.

DVDD V = potentiometer value for adjusting onboard DVDD regulator

Arm DVDD = only when 1 is written to this, DVDD regulator is set,

When 0 is written, DVDD voltage is not changed.

When read register

D31-24	D23-12	D11-0
-	DVDD I	AVDD I

AVDD I = AVDD onboard regulator current

DVDD I = DVDD onboard regulator current

< Read current DVDD33 >

Address = 0x4

D31-12	D11-0
-	DVDD33 I

DVDD33 I = DVDD33 onboard regulator current

< Set voltage/read current PWELL and SUB >

Address = 0x5

When write to register

D31-22	D21	D20-D11	D10	D9-0
-	Arm SUB	SUB V	Arm PWELL	PWELL V

PWELL V = potentiometer value for adjusting onboard PWELL regulator

Arm PWELL = only when 1 is written to this, PWELL regulator is set,

When 0 is written, PWELL voltage is not changed.

SUB V = potentiometer value for adjusting onboard SUB regulator

Arm SUB = only when 1 is written to this, SUB regulator is set,

When 0 is written, SUB voltage is not changed.

When read register

D31-24	D23-12	D11-0
-	SUB I	PWELL I

PWELL I = PWELL onboard regulator current

SUB I = SUB onboard regulator current

< Short combination AGND, PWELL and SUB >

Address = 0x06

D31-4	D3-2	D1-0
-	SUB SHORT	PWELL SHORT

PWELL SHORT = select voltage to PWELL pad.

0 = AGND, 1 = PWELL regulator, 2 = SUB regulator

SUB SHORT = select voltage to SUB pad.

0 = AGND, 1 = PWELL regulator, 2 = SUB regulator

< Read DACMONV and DACMONI voltage >

Address = 0x7

D31-24	D23-12	D11-0
-	DAVMONI V	DACMONV V

DACMONV V = voltage on DACMONV pad

DACMONI V = voltage on DACMONI pad

< Operation mode >

Address = 0x8

D31-2	D1-0
-	MODE

MODE = operation mode

0 = Manual contact test mode. Probing and sensing should be done by software.

1 = Auto contact test mode. Probing and sensing are done by USB3 MCU. Contact status is shown on LEDs. Status or pin contact results can be read by software too.

2= DAQ mode. Probecard is electically disconnected from ASIC chip and all signals are passed to DAQ board.

3= Standalone mode. Same as DAQ mode except voltage to chip is supplied by probecard rather than by PCI connector. (Full standalone DAQ functionality will be implemented later.)

< DATA and BUSY buffer direction >

Address = 0x9

D31-5	D4	D3	D2	D1	D0
-	BUSY DIR	DATA3 DIR	DATA2 DIR	DATA1 DIR	DATA0 DIR

DATAx DIR = set DATA direction. 0 = ASIC chip -> DAQ, 1=DAQ -> ASIC chip

BUSY DIR = set BUSY direction. 0 = ASIC chip -> DAQ, 1=DAQ -> ASIC chip

< Start ADC conversion for monitoring>

Address = 0xA

D31-D7	D6	D5	D4	D3	D2	D1	D0
-	DACMONI S	DACMONV S	SUB S	PWELL S	DVDD33 S	DVDD S	AVDD S

This one should be written before reading current/voltage monitoring ADC.

AVDD S = when 1 start ADC conversion to measure AVDD regulator current

DVDD S = when 1 start ADC conversion to measure DVDD regulator current

DVDD33 S = when 1 start ADC conversion to measure DVDD33 regulator current

PWELL S = when 1 start ADC conversion to measure PWELL regulator current

SUB S = when 1 start ADC conversion to measure SUB regulator current

DACMONV S = when 1 start ADC conversion to measure DACMONV voltage.

DACMONI S = when 1 start ADC conversion to measure DACMONI voltage.

< LED >

Address = 0xB

D31-8	D7	D6	D5	D4	D3	D2	D1	D0
-	LED7	LED6	LED5	LED4	LED3	LED2	LED1	LED0

When operation mode = 1 (Auto contact test mode), LEDs represent following status :

LED0 : All AVDD pads are contacted.

LED1 : All DVDD pads are contacted.

LED2 : All AVSS pads are contacted.

LED3 : All DVSS pads are contacted.

LED4 : DVDD33 pad is contacted.

LED5 : All PWELL pads are contacted.

LED6 : All SUB pads are contacted.

LED7 : All signal pads are contacted.

For other mode, LED is user programmed register.

<Clock synthesizer lock status>

Address = 0xC

D31-1	D0
-	LOCK STATUS

LOCK STATUS = 1 is read when clock synthesizer is locked, otherwise 0 is read.

<Dummy register for readback test>

Address = 0xF

D31-0
READBACK

READBACK = dummy register for debugging purpose

3. Library functions.

Probecard's vendor ID is 0x0547 and product ID is 0xCE03. It uses end point 2 for input and end point 6 for output.

```
void PC3_USBinit(libusb_context **ctx)
```

Initialize libusb. If other process uses libusb library, different context should be used.

This function should be called before calling any other functions.

```
void PC3_USBexit(libusb_context *ctx)
```

Deinitialize libusb.

```
void PC3_Open(int sid, libusb_context *ctx)
```

Open probecard. Sid is serial ID programmed to probecard. When using multiple probecards sid should be uniquely assigned to each probecard.

```
void PC3_Close(int sid)
```

Close probecard.

```
int PC3_Receive(int sid, unsigned int count, unsigned int addr,
                unsigned char *data)
```

Read from probecard.

count : number of 32 bit words to be read.

addr : starting address to be read. Address is auto-incremented.

data : character array for readout data

```
int PC3_Write(int sid, unsigned int addr, unsigned int data)
```

Write register in probecard.

addr : register address to be written.

data : data to be written.

```
unsigned int PC3_Read(int sid, unsigned int addr)
```

Read register.

addr : address to be read.

```
void PC3_Write_PWR(int sid, int avss, int avdd, int dvss, int dvdd, int dvdd33,
                   int pwell, int sub)
```

When write 1 to bit, CMOS probe signal is activated.

avss = 0 ~ 0xFF, avdd = 0 ~ 0xFF, dvss = 0 ~ 0xFF, dvdd = 0 ~ 0xFF, dvdd33 = 0 or 1

pwell = 0 ~ 0xF, sub = 0 ~ 0x1F

```
void PC3_Read_PWR(int sid, int *avss, int *avdd, int *dvss, int *dvdd, int *dvdd33,
                  int *pwell, int *sub)
```

Returns 1 when contact is made, 0 when contact is not made.

avss = 0 ~ 0xFF, avdd = 0 ~ 0xFF, dvss = 0 ~ 0xFF, dvdd = 0 ~ 0xFF, dvdd33 = 0 or 1

pwell = 0 ~ 0xF, sub = 0 ~ 0x1F

```
void PC3_Write_SIG(int sid, int data, int mistral_o, int busy, int ctrl, int chipid,
                   int por_dis, int rst_n, int hsdata, int dclk, int dctrl, int mclk,
                   int dacmonv, int dacmoni)
```

When write 1 to bit, CMOS probe signal is activated. For LVDS signals, MSB is negative pad, LSB is positive pad.

data = 0 ~ 0xF, mistral-o = 0 ~ 7, busy = 0 or 1, ct기 = 0 or 1, chipid = 0 ~ 0x7F,

por_dis = 0 or 1, rst_n = 0 ~ 1, hsdata = 0 ~ 3, dclk = 0 ~ 3, dctrl = 0 ~ 3, mclk = 0 ~ 3,

dacmonv = 0 or 1, dacmoni = 0 or 1

```
void PC3_Read_SIG(int sid, int *data, int *mistral_o, int *busy, int *ctrl, int *chipid,
                  int *por_dis, int *rst_n, int *hsdata, int *dclk, int *dctrl,
                  int *mclk, int *dacmonv, int *dacmoni)
```

Returns 1 when contact is made, 0 when contact is not made. For LVDS signals, MSB is negative pad, LSB is positive pad.

data = 0 ~ 0xF, mistral-o = 0 ~ 7, busy = 0 or 1, ct기 = 0 or 1, chipid = 0 ~ 0x7F,

por_dis = 0 or 1, rst_n = 0 ~ 1, hsdata = 0 ~ 3, dclk = 0 ~ 3, dctrl = 0 ~ 3, mclk = 0 ~ 3,

dacmonv = 0 or 1, dacmoni = 0 or 1

```
void PC3_Write_V(int sid, int avdd, int dvdd, int pwell, int sub, float avdd_value,
                 float dvdd_value, float pwell_value, float sub_value)
```

Set voltage to onboard regulators. This can be effective only when operation mode is standalone mode.

avdd = when 1 sets AVDD voltage regulator, when 0 regulator setting is ignored.

dvdd = when 1 sets DVDD voltage regulator, when 0 regulator setting is ignored.

pwell = when 1 sets PWELL voltage regulator, when 0 regulator setting is ignored.

sub = when 1 sets SUB voltage regulator, when 0 regulator setting is ignored.

avdd_value = AVDD regulator voltage in V.

dvdd_value = AVDD regulator voltage in V.

pwell_value = AVDD regulator voltage in V.

sub_value = AVDD regulator voltage in V.

```
void PC3_Read_I(int sid, float *avdd, float *dvdd, float *dvdd33, float *pwell,
                float *sub, float *dacmonv, float *dacmoni)
```

Read currents of voltage regulators and DACMONV, DACMONI voltage.

avdd = AVDD regulator current in mA.

dvdd = DVDD regulator current in mA.

dvdd33 = DVDD33 regulator current in mA.

pwell = PWELL regulator current in mA.

sub = SUB regulator current in mA.

dacmonv = DACMONV pad voltage in V.

dacmoni = DACMONI pad voltage in V.

void PC3_Write_SHORT(int sid, int pwell, int sub)

Write voltage output combination to PWELL and SUB pads.

pwell = set PWELL pad voltage. 0 = AGND, 1 = PWELL regulator, 2 = SUB regulator

subl = set SUB pad voltage. 0 = AGND, 1 = PWELL regulator, 2 = SUB regulator

void PC3_Write_MODE(int sid, int value)

Write operation mode.

Value : 0 = Manual contact test mode, 1 = Auto contact test mode

2 = DAQ mode, 3 = Standalone mode

void PC3_Write_DIR(int sid, int data, int busy)

Write CMOS bidirectional signal direction. When 1 direction is DAQ->ASIC, when 0 ASIC->DAQ.

data = DATA lines. 0 ~ 0xF

busy = BUSY line. 0 ro 1

void PC3_Write_LED(int sid, int value)

Write LEDs. When bit is 1 LED is turned ON. 0 for OFF.

unsigned int PC3_Read_STATUS(int sid)

Read status of LED. In auto contact test mode it shows contact test status.

unsigned int PC3_Read_LOCK(int sid)

Read clock synthesizer locking status. Returns 1 when synthesizer is locked, otherwise returns 0.

void PC3_Write_READBACK(int sid, unsigned int data)

Write readback register. 0 ~ 0xFFFFFFFF

unsigned int PC3_Read_READBACK(int sid)

Read readback register.

4. Sample user code

   This one can be found in probecardv3/software/run. Please set library path by

   source run_probecard_v3.sh

   Use make to compile run_probecard_v3.c